

VisionEngine $\rightleftharpoons$ autonomous-QA vision-analysis integration assessment

- Date: 2026-07-02
 - Scope: read-only assessment + integration plan. No source was edited, no submodule was init/deinit'd, nothing committed.
 - Subject submodule: `submodules/vision_engine` (`.gitmodules` → `git@github.com:vasic-digital/VisionEngine.git`, Go module `digital.vasic.visionengine`).
 - Consumer under study: `scripts/autonomous-qa/vision-analyze.sh` + its caller chain (`scripts/autonomous-qa/run-iteration.sh`).
 - Vocabulary note (§11.4.6): every capability below is stated as FACT from the actual files read. Items not present in the docs/source are marked UNCONFIRMED:.
-

1. Capability summary — what VisionEngine DOES

VisionEngine is a Go computer-vision + LLM-vision toolkit for UI analysis and navigation-graph construction (README.md:1-30). Four cooperating layers:

- **pkg/analyzer** — Analyzer + VideoProcessor interfaces; value types (UIElement, ScreenAnalysis, ScreenDiff, Rect, Size, TextRegion, VisualIssue, ScreenIdentity, Action, KeyFrame); StubAnalyzer reference impl. `AnalyzeScreen(nil)` returns the sentinel `analyzer.ErrEmptyScreenshot` (README.md:88-90).
- **pkg/graph** — NavigationGraph (directed screen-transition graph, BFS `PathTo`, `Coverage()`, `UnvisitedScreens()`) with DOT/JSON/Mermaid exporters (README.md:64-75, ARCHITECTURE.md:71-94). This is the layer "most-imported by HelixQA".
- **pkg/llmvision** — VisionProvider interface + 8 adapters (OpenAI GPT-4o, Anthropic Claude, Gemini, Qwen-VL, Kimi/Moonshot, StepFun, Astica, Ollama) + FallbackChain composer

(README.md:21-24, ARCHITECTURE.md:19-45). Core method:
provider.AnalyzeImage(ctx, imageBytes, prompt) (string, error)
(USER_GUIDE.md:58).

- **pkg/opencv** — mechanical CV (element detection, SSIM comparison, color analysis, feature/ORB, video). Build-tag gated: **default build ships stubs; real GoCV bindings only under -tags vision** (README.md:28-30, factory.go:4 //go:build !vision). All gocv.io/x/gocv imports are //go:build vision-tagged (verified across pkg/opencv/*_vision.go).
- **Support layers:** pkg/config (env loader), pkg/i18n (Translator seam), pkg/remote (VisionPool / SSH deploy of Ollama + llama.cpp RPC workers).

The intended pipeline (ARCHITECTURE.md:55-69) is two-layer: Layer 1 = GoCV mechanical (element detection / SSIM / color); Layer 2 = LLM vision (screen identification / UI comprehension / issue detection), combined.

2. Interface / IO — how a consumer drives it

There are **two consumption modes**, both real (no cloud dashboard, no hosted service):

2.a CLI bridge — cmd/visiondescribe (the relevant one for the QA loop)

cmd/visiondescribe/main.go is a "thin, project-agnostic CLI bridge that turns a PNG screenshot into a structured ... UI description by calling a REAL vision-capable LLM" (main.go:4-30). This is the single command entrypoint (find cmd → only cmd/visiondescribe/main.go).

- **Input:** -image <path.png> (a single PNG screenshot), optional -focus "<hint>" appended to the prompt, -provider flag present in the usage banner, and -probe (report configured providers and exit).
- **Output:** a single JSON envelope on stdout (main.go:68-93). Fields include ok, provider, model, image_bytes, latency_ms, raw_text_len, parsed_ok, raw_text, error, operator_block, and a typed result object:
 - app_label, app_pkg_guess, screen_context (one of launcher, app_home, content_list, player, settings, dialog, sign_in, error, loading, unknown), visible_title, attributes[] (short

visible facts, e.g. "progress bar 35%"), `playback_state` (playing/paused/buffering/stopped/not_applicable), `overlays[]` (dialog/toast/ad/paywall/ANR/sign-in/geo-block text), `full_description` (main.go:50-93).

- **Exit codes** (main.go:28-29, 106-170): 0 = described; 2 = bad input / model error / empty (0-byte) image; 3 = no vision provider configured (OPERATOR-BLOCKED — emits a structured envelope, never a canned description); 4 = vision call failed (provider unreachable).
- **Anti-bluff provenance** (main.go:17-21): each success emits resolved provider name, model id, image byte count, wall-clock latency, and raw-text length — fields a stub cannot forge because they come from the live HTTP round-trip. Empty/FLAG_SECURE screenshots are caught explicitly (main.go:120-123).

FACT — provider selection in the shipped CLI is **cloud-only**: `buildProvider()` (main.go:176-250) wires only `anthropic > openai > gemini > openrouter`, in that priority. **Ollama is NOT wired into `cmd/visiondescribe`** (`grep -c ollama cmd/visiondescribe/main.go = 0`), even though `pkg/llmvision/ollama.go` exists as a library adapter. This is load-bearing for the offline question (§3).

2.b Go library

For a custom driver: `config.LoadFromEnv()` → `llmvision.NewGeminiProvider(...)/NewOllamaProvider(...)/NewFallbackProvider(...)` → `provider.AnalyzeImage(ctx, bytes, prompt)` (USER_GUIDE.md:51-70). The `pkg/opencv` mechanical layer and `pkg/graph` are library-only (no CLI exposes them). `pkg/remote.VisionSlot.Lock()/Unlock()` serializes vision calls per device (USER_GUIDE.md:98-114) — directly relevant to the QA matrix runner's per-device contention (§11.4.119).

There is **no HTTP server mode** and **no prebuilt binary** shipped — the consumer builds from source (`go build ./cmd/visiondescribe/`).

3. Runtime needs & offline posture (§6.H / §11.4.10)

Build:

- `cmd/visiondescribe` builds **without OpenCV / without cgo** — it imports only `pkg/config + pkg/llmvision` (which imports only `pkg/i18n`); no `gocv` in that path (verified: every `gocv` import is `//go:build vision`). So `go build ./cmd/visiondescribe/` is pure-Go.
- Toolchain: `go.mod` declares `go 1.25.3`. **GAP:** `Dockerfile.vision` bases on `golang:1.24-bookworm` (`Dockerfile.vision:5`) — `1.24 < 1.25.3`. `Dockerfile.vision` is the OpenCV image (`go build -tags vision ./...`); building the pure-Go `visiondescribe` there would also hit the version floor. A 1.25.3+ toolchain (host or a corrected container per §11.4.76 Containers submodule) is required.
- The full `-tags vision` OpenCV path needs `build-essential cmake pkg-config libopencv-dev libopencv-contrib-dev` (`Dockerfile.vision:8-14`) + `cgo`.

Can it run offline / does data leave the host? — nuanced, three cases:

Path	Offline?	Data leaves host?	CLI today?
<code>visiondescribe</code> → <code>anthropic/openai/gemini/openrouter</code>	NO (cloud)	YES — screenshot bytes POSTed to a cloud vision API	YES (shipped)
<code>pkg/llmvision</code> Ollama adapter (local/LAN model, e.g. <code>minicpm-v:8b</code> at <code>HELIX_OLLAMA_URL</code>)	YES if Ollama runs on localhost/LAN	NO (stays on host/LAN)	NO — library only; not wired into visiondescribe
<code>pkg/opencv</code> mechanical (SSIM/element/color) <code>-tags vision</code>	YES (fully local)	NO	NO — library only, no CLI

Conclusion: the **shipped CLI's real-provider path is cloud** — using it as-is sends recorded QA frames to a third-party vision API, a §6.H / §11.4.10 data-egress consideration. A fully-offline VisionEngine oracle is achievable but requires either (a) a small code extension wiring the

existing Ollama adapter into `buildProvider()` (upstream, per §11.4.74 extend-don't-reimplement), or (b) a custom Go main over `pkg/opencv` mechanical CV. Both are additive work, not shipped today.

Config env vars (`pkg/config.go`, `.env.example`): `HELIX_VISION_PROVIDER` (auto/named), `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `GOOGLE_API_KEY`/`GEMINI_API_KEY`, `OPENROUTER_API_KEY`, `HELIX_OLLAMA_URL` + `HELIX_OLLAMA_MODEL`, per-provider model overrides, `HELIX_VISION_TIMEOUT`, `HELIX_VISION_MAX_IMAGE_SIZE`, `HELIX_VISION_SSIM_THRESHOLD`, `HELIX_VISION_OPENCV_ENABLED`. Keys load via env only (no hardcoding — aligns with §6.R).

4. Current vision-analysis flow (what exists today)

`scripts/autonomous-qa/vision-analyze.sh` (a self-contained bash script) is the current vision layer. It is **local, deterministic, offline, and zero-cost** — it does NOT use VisionEngine.

- **Inputs** (per iteration `<iter>/raw/`, produced by `run-iteration.sh`): chunked screenrecord mp4s (`rec_*.mp4` / `qa_rec_*.mp4`), `logcat.txt`, `gradle-connected.log`, plus a best-effort `final-screen.png` still (screenshot, `run-iteration.sh:95`), and optional `verdict.json` cross-ref.
- **Analyses**: `ffmpeg signalstats` for blank/white/black/monochrome frame detection; `ffmpeg freezedetect` for stuck/frozen spans; tesseract OCR (ONLY if present) to confirm expected affordance tokens (download, magnet, torrent, seed, size default) and to catch stuck-loading; `logcat grep` for `FATAL EXCEPTION/E AndroidRuntime/ANR in`.
- **Outputs**: a curated `vision-analysis.md` per iteration + a cycle rollup table; exit codes 0=CLEAN, 1=DEFECTS-FOUND, 2=CANNOT-ANALYZE/INCOMPLETE/TOOL-MISSING.
- **Anti-bluff posture (already strong)**: never prints CLEAN for analysis it didn't perform — `ffmpeg` missing → exit 2; no recordings → exit 2; unreadable → INCOMPLETE (never CLEAN); tesseract absent → OCR checks SKIPPED and said so, no fabricated screen-content claims.

FACT — `vision-analyze.sh` has **no current caller inside scripts/** (`grep -rn vision-analyze scripts/` finds only the file itself). It is presently a standalone/manually-invoked tool. The user-visible-outcome signal that IS wired into the loop is a **logcat marker**: `run-`

iteration.sh:113-136 treats C70-RESULT ... DOWNLOAD-OK (logged by Challenge70 only after the on-screen download/magnet affordance is confirmed) as the hard evidence the real download flow was reached, and uses it in the anti-bluff PASS-via-marker decision table (run-iteration.sh:164-206). DOWNLOAD-OK is thus the existing "results-rendered + download-affordance-visible" contract; today it is asserted by the Compose test in-process, not by pixel analysis of the recording.

The gap VisionEngine can close

The current DOWNLOAD-OK signal is an **in-process assertion** (the test says it saw the affordance). The recorded frames are analyzed only by the deterministic ffmpeg/OCR floor. There is no **independent semantic reading of the actual pixels** confirming "search results are rendered" / "a download/magnet affordance is visible" / "no blocking overlay". That independent oracle is exactly what visiondescribe produces (screen_context=content_list, attributes contains download/magnet, overlays=[], playback_state).

5. Proposed integration — additive second oracle, not a replacement

Keep vision-analyze.sh's deterministic ffmpeg/OCR floor as the always-on, offline, zero-cost baseline (it already satisfies §11.4.107-style freeze/frame-advance + OCR). **ADD** an optional VisionEngine semantic layer that reads the same recorded frames and emits a structured, machine-readable, user-visible-outcome verdict that composes with the §6.AK cycle-coverage gate as a second, independent DOWNLOAD-OK / results-rendered proof.

5.a Wiring steps (concrete)

1. **Build the CLI once, cached** (per §11.4.82 persistent caches; do not rebuild per iteration):

```
( cd submodules/vision_engine && go build -o "$CACHE/  
visiondescribe" ./cmd/visiondescribe/ )
```

Guard on Go $\geq$ 1.25.3; if absent, run the build inside a Containers-submodule Go image (§11.4.76) with a corrected base (see §6 gap on Dockerfile.vision:5).

2. **Add a bounded VisionEngine block to vision-analyze.sh** (new, opt-in via `--vision-engine <bin> + --vision-focus "<hint>"`, default OFF so the offline floor never regresses). Per analyzed recording, select representative frames already extracted at `<iter>/raw/.vision-frames/<base>/f_*.png` (reuse the 1fps PNGs the script already produces — no new extraction) plus `raw/final-screen.png`. Run:

```
"$VE_BIN" -image "$frame" \
  -focus "Are search results rendered with a visible download
          or magnet affordance? Report any blocking overlay." \
  > "$fdir/ve_$(basename "$frame").json" 2>/dev/null; rc=$?
```

3. **Interpret the envelope into a verdict signal** (jq or the existing awk/grep style):

- `rc==0` && `result.screen_context` ∈ {`content_list`, `player`, `app_home`} AND (download/magnet/torrent token present in `result.attributes` OR `result.visible_title`) AND `result.overlays` == [] → VE-DOWNLOAD-OK (semantic confirmation).
- `rc==0` && `overlays` non-empty (`paywall/sign_in/error/ANR/geo`) → VE-OVERLAY-BLOCKING defect (a §6.AB non-crashing defect class the ffmpeg floor cannot name).
- `rc==0` && `screen_context` ∈ {`loading`, `error`} with no affordance → VE-WRONG-SCREEN / VE-STUCK defect.
- `rc==3` (OPERATOR-BLOCKED, no key) → record VE-SKIPPED: no vision provider configured — **treated as SKIP, never CLEAN/PASS** (matches the script's existing tesseract-absent posture and §11.4.69 no-fail-open-skip).
- `rc` ∈ {2,4} → VE-INCOMPLETE (unreachable/bad image), never CLEAN.

4. **Emit into the curated vision-analysis.md** as a distinct "VisionEngine semantic oracle" section, and add the verdict token to the same defects/verdict machinery the script already has. The script's tri-state (CLEAN/DEFECTS-FOUND/INCOMPLETE) is unchanged; VisionEngine can only ADD defects or an independent confirmation — it can never downgrade a deterministic defect to CLEAN.

5. **Compose with §6.AK** (`scripts/check-cycle-coverage.sh` + the `cycle-coverage-map-<ver>.yaml` under `.lava-ci-evidence/distribute-changelog/...`): for a CHANGELOG claim like "search results / download works", the covering Challenge's device evidence can now carry TWO independent user-visible proofs — the in-process C70-RESULT DOWNLOAD-OK logcat marker AND the

VisionEngine VE-DOWNLOAD-OK semantic verdict on the recorded pixels. Record the VisionEngine verdict path in the per-iteration evidence so the coverage-intersection gate can cite pixel-level confirmation, not only the in-process assertion.

5.b Where it slots in the caller chain

`vision-analyze.sh` is currently uncalled from `scripts/`. Recommended: invoke it from the iteration path (after `run-iteration.sh` writes `raw/`) or from the cycle rollup, with the VisionEngine block enabled only when a vision provider (or local Ollama) is configured. The device serial/VisionSlot serialization (§11.4.119 / `USER_GUIDE.md:98-114`) matters if the matrix runs providers concurrently — serialize vision calls per device.

5.c Dependencies to add

- Go $\geq$ 1.25.3 toolchain (host or Containers-submodule image).
 - A vision provider credential in the credential single-source-of-truth (ANTHROPIC_API_KEY | OPENAI_API_KEY | GOOGLE_API_KEY | OPENROUTER_API_KEY) OR — for offline — a local Ollama endpoint plus the CLI extension in §6.
 - `jq` recommended for envelope parsing (optional; `grep/awk` works).
 - No new git remote (the submodule is already declared in `.gitmodules`).
-

6. Risks & gaps (§11.4.6 — stated as fact or marked unconfirmed)

1. **Cloud egress (§6.H / §11.4.10).** The shipped `visiondescribe` real path is cloud-only (`buildProvider` `main.go:176-250` wires `anthropic/openai/gemini/openrouter`; `grep -c ollama = 0`). Using it sends recorded QA frames off-host. For a truly-offline oracle, extend `buildProvider()` upstream to wire the existing `pkg/llmvision/ollama.go` adapter (§11.4.74), or build a custom main over the `pkg/opencv` mechanical layer. **This is the biggest integration gap.**
2. **Non-determinism (§11.4.50).** LLM output varies run-to-run; a semantic verdict cannot be the sole gate. Mitigation: keep the deterministic `ffmpeg/OCR` floor authoritative; use VisionEngine as a corroborating oracle and require conservative `screen_context+overlay` checks, not free-text.

3. **Toolchain version gap.** `Dockerfile.vision:5` (`golang:1.24-bookworm`) is below `go.mod`'s `go 1.25.3` (`go.mod:3`) — the shipped Docker recipe would fail to build. A corrected Go image is needed for containerized builds (§11.4.76).
4. **No prebuilt binary / no server mode.** Consumer must compile `cmd/visiondescribe` from source; there is no daemon to keep warm. Cache the built binary (§11.4.82).
5. **OpenCV mechanical layer is library-only.** `pkg/opencv` (SSIM/element/color — the offline mechanical Layer 1) has no CLI and needs `-tags vision + OpenCV + cgo`. Exposing it would be additive upstream work.
6. **OPERATOR-BLOCKED must map to SKIP, never PASS.**
`visiondescribe` exit 3 (no key) is honest; the integration MUST record it as SKIP-with-reason and NOT let a missing provider silently produce a CLEAN verdict (§11.4.69 no-fail-open-skip; mirrors the script's tesseract-absent handling).
7. **Submodule availability.** `.gitmodules` declares `submodules/vision_engine` (path appears in the parent index; UNCONFIRMED whether the gitlink is fully committed vs staged — parent git status showed it flagged as an addition). The integration presumes the submodule is initialized/checked out before the build; guard for absence.
8. **pkg/graph opportunity (not required).** `NavigationGraph + Coverage()/UnvisitedScreens()` could later map QA exploration coverage, but it is orthogonal to the DOWNLOAD-OK signal and out of scope for this integration.
9. **Export mandate (§11.4.65).** This `.md` is owed synchronized `.html/.pdf` siblings under the Lava export discipline; not generated here (read-only assessment). Owed as a follow-up.